EnergyWatch AI

DOCUMENTATION TECHNIQUE

Technical Documentation

Architecture, agents, observability, evaluation, deployment and course mapping of the EnergyWatch AI platform.

Projet	**EnergyWatch AI, plateforme multi-agents (LangGraph)**
Secteur	**Énergie**
Cadre	**Mini-projet S5 · Agent en production**
Version	**1.2.0**
Date	**30 octobre 2026**
Dépôt	**github.com/bouincha/Energy-Watch-AI**

Table des matières

1. Overview
2. Architecture
3. Dashboard screenshots
4. Evaluation & Benchmarking
5. Course mapping: where each week's concepts live in the code
6. Deployment Guide
7. Put EnergyWatch AI online with Cloudflare Tunnel (free, no card)

Overview

🎯 The problem

A company runs four very different sites in Morocco. Each one wastes energy in its own way, and nobody has time to read 16 sub-meters every hour:

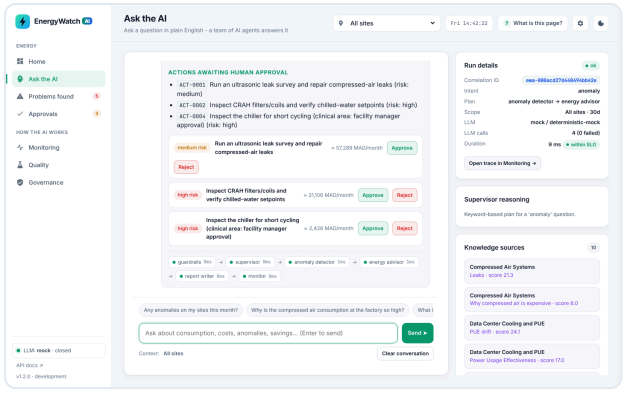
Site	Type	Typical waste
Casablanca Headquarters (`HQ-CASA`)	Office, 12,000 m²	HVAC and lighting left on at night / weekends
Kenitra Assembly Plant (`PLANT-KEN`)	Factory + rooftop solar	Compressed-air leaks
Rabat Data Center (`DC-RABAT`)	Data center	Cooling degradation (PUE drift)
Marrakech Clinic (`CLINIC-MRK`)	Healthcare, 24/7	Chiller faults

The simulated dataset (30 days × 16 meters, hourly) contains **5 realistic injected anomalies** worth **≈ 24,000 MAD** of avoidable cost, which the system has to find, explain and fix.

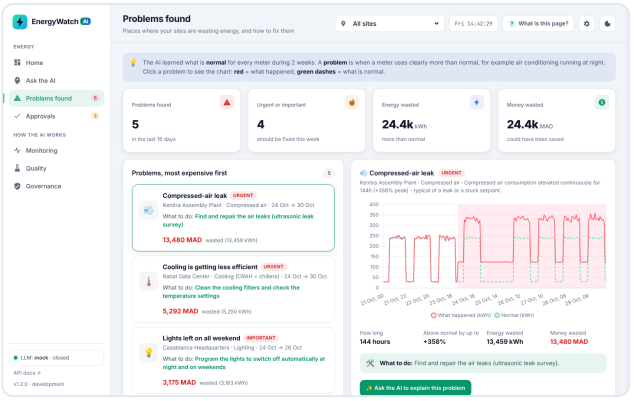
✨ What EnergyWatch AI does

- 💬 **Answers questions in natural language**, e.g. *"Any anomalies this month?"*, *"How can I reduce the data center cooling costs?"*, *"Give me the full monthly report"*.
- 🔍 **Detects anomalies** statistically (robust z-score against a per-hour, per-day-type baseline) and classifies them: night operation, weekend operation, sustained increase, drift, spike. It finds 5/5 injected anomalies with 0 false positives across 210 seed/calendar combinations.
- 📚 **Recommends actions grounded in a knowledge base** (RAG over 7 energy-efficiency guides) with citations and estimated monthly savings.
- 🧑‍⚖️ **Human-in-the-loop**: agents only *propose* actions. Low-risk actions are auto-approved by policy, while medium and high-risk actions wait for an operator. Every decision is written to an append-only audit trail.
- 🛡️ **Guardrails** (OWASP LLM Top 10): prompt injection, secret leakage, off-topic requests, rate limiting and read-only tools.
- 📡 **Monitoring agent**: every run gets a **correlation ID** that ties together the HTTP header, JSON logs, agent spans, metrics, incidents and audit entries.
- 🚨 **Incident management**: a circuit breaker, a degraded mode, 4 runbooks, and incidents that open and resolve automatically, with an MTTR for each.
- 🖥️ **A dashboard written for people, not engineers**: Home (one-sentence summary and an energy health score), Ask the AI, Problems found (each with "what to do"), Approvals, Monitoring, Quality and Governance. A **"What is this page?"** button explains every page in plain words.
- ✅ **Quality page**: shows in one place whether everything is valid. It displays the latest GitHub Actions run (every job and step), runs the evaluation quality gates live, and runs self-checks on the instance itself.

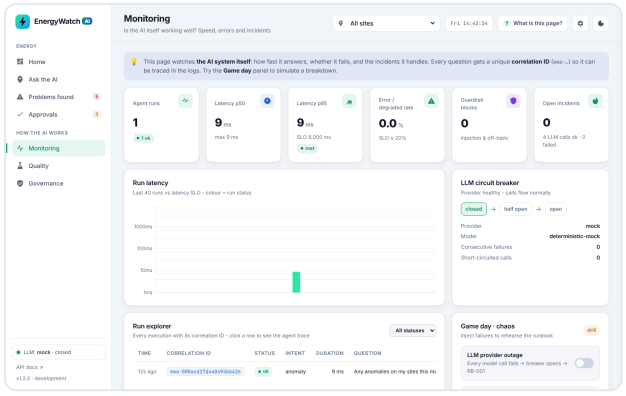
AI Assistant



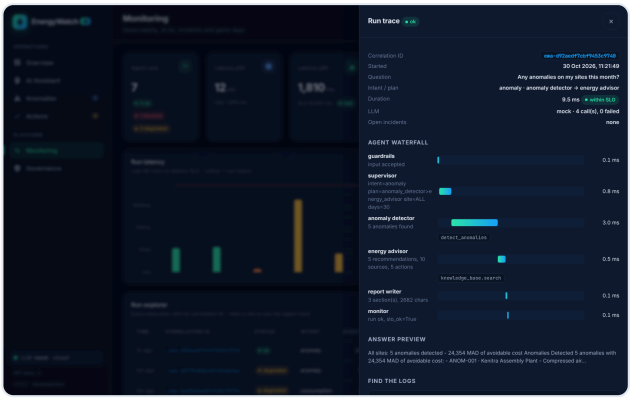
Anomalies



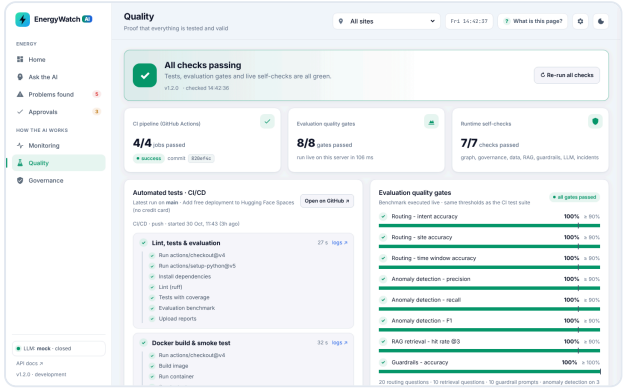
Monitoring & incidents



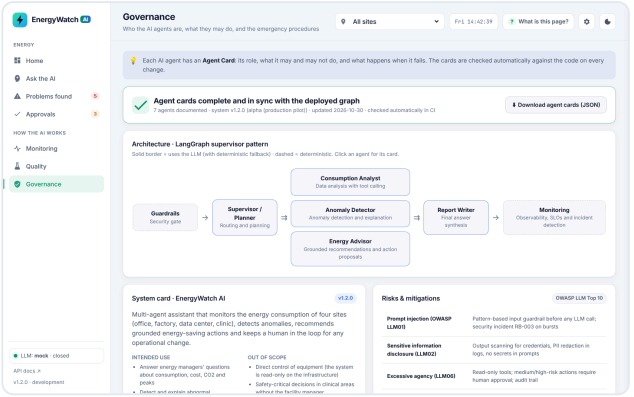
Run trace (correlation ID)



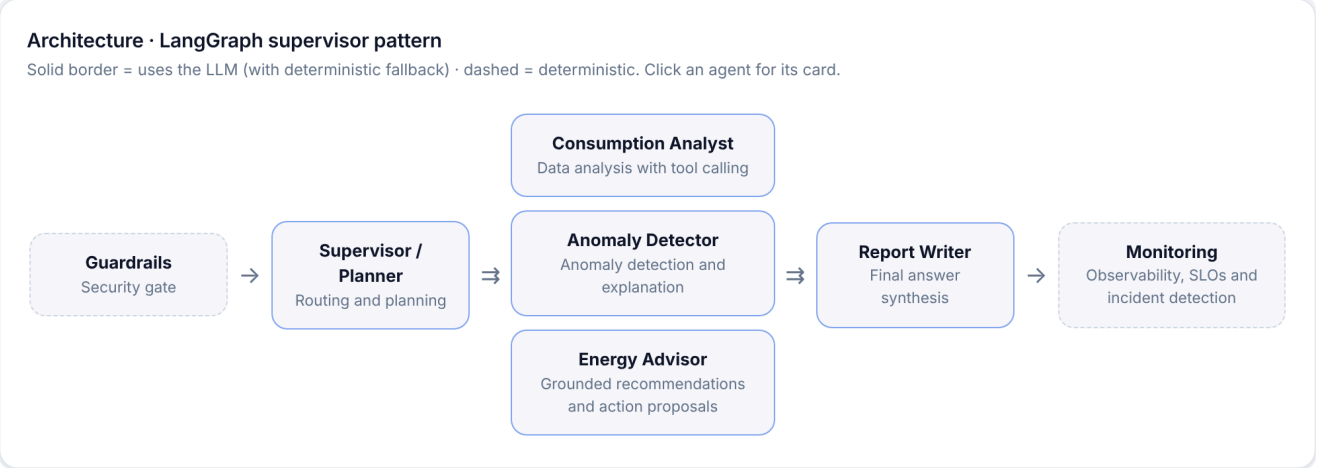
Quality: is everything valid?



Governance



🏗 Architecture



Architecture of the LangGraph multi-agent system (screenshot of the Governance page)

- **LangGraph** `StateGraph` with a *supervisor / planner* pattern. The supervisor returns a Pydantic-validated plan, and a dispatcher runs the planned specialists in order.
- **Deterministic tools compute the numbers**; the LLM only plans, chooses tools and writes. This stops it from inventing figures.
- **Every LLM agent has a deterministic fallback**, so the system still answers, in *degraded* mode, when the model is down.
- **Pluggable LLM**: Groq, Gemini, OpenAI or Anthropic, plus a **deterministic mock** that needs no API key (used for tests, CI and offline demos).

Details: docs/ARCHITECTURE.md

✅ Validation criteria → where to find them

Criterion	Implementation	Proof
Observability: correlation ID on every run (monitoring agent)	`app/core/observability.py`, `traced()` in `app/agents/nodes.py`, `monitor_node`	`X-Correlation-ID` header, JSON logs, Monitoring page → Run explorer, `tests/test_graph.py::test_every_run_has_a_correlation_id_on_every_span`
Tests: suite runs successfully	70 tests (unit, integration, end-to-end, evaluation gates), 95 % coverage	GitHub Actions → *CI/CD* workflow, and the dashboard's **Quality** page
Governance: complete and up-to-date Agent Card	`app/governance/agent_cards.json` (served at `/api/agent-card`) + docs/AGENT_CARD.md	CI test fails if a card is missing a field or drifts from the graph or version
Containerization & deployment: agent accessible via a deployed API	`Dockerfile` (multi-stage, non-root, healthcheck), `render.yaml`	docs/DEPLOYMENT.md, `scripts/smoke_test.py`

Criterion	Implementation	Proof
Incident: runbook applied, incident handled	`app/core/incidents.py` (RB-001 → RB-004), chaos controller	docs/RUNBOOK.md, docs/INCIDENT_REPORT_INC-0001.md
Technical docs + demo video	`docs/`	docs/DEMO_VIDEO_SCRIPT.md

How each course week (S1 → S5) maps to the code: docs/COURSE_MAPPING.md

🚀 Quick start

Requirements: Python 3.11+ (3.12 recommended) and Git. Docker is optional.

```
git clone https://github.com/bouincha/Energy-Watch-AI.git
cd Energy-Watch-AI

python -m venv .venv
# Windows:      .venv\Scripts\activate
# macOS/Linux:  source .venv/bin/activate

pip install -r requirements-dev.txt
uvicorn app.main:app --reload
```

Open **http://localhost:8000** for the dashboard, and **http://localhost:8000/docs** for the interactive API documentation.

> No API key is needed: without one, the app runs with the deterministic mock LLM.

With Docker

```
docker build -t energywatch-ai .
docker run -p 8000:8000 energywatch-ai
# or: docker compose up --build
```

🤖 Using a real LLM

Copy `.env.example` to `.env` and set **one** key. The provider is picked automatically:

Provider	Variable	Free tier	Default model
Groq	`GROQ_API_KEY`	✅	`llama-3.3-70b-versatile`
Google Gemini	`GOOGLE_API_KEY`	✅	`gemini-2.5-flash`
OpenAI	`OPENAI_API_KEY`	❌	`gpt-4o-mini`
Anthropic	`ANTHROPIC_API_KEY`	❌	`claude-haiku-5-5`

Override the model with `LLM_MODEL=...`. You can check the active provider in the sidebar of the dashboard or at `GET /ready`.

🔌 API

Method	Endpoint	Purpose
`POST`	`/api/agent/ask`	Run the multi-agent graph (`{"query": "...", "site_id": null}`)
`GET`	`/api/runs` , `/api/runs/{correlation_id}`	Run explorer and full trace
`GET`	`/api/monitoring/summary`	Latency p50/p95, error rate, per-agent stats, breaker state
`GET`	`/metrics`	Prometheus metrics
`GET`	`/api/overview` , `/api/sites` , `/api/anomalies` , `/api/live`	Energy data
`GET/POST`	`/api/actions` , `/api/actions/{id}/approve|reject`	Human-in-the-loop
`GET`	`/api/audit`	Audit trail
`GET/POST`	`/api/incidents` , `/api/incidents/{id}/resolve` , `/api/runbooks`	Incident management
`GET/POST`	`/api/chaos`	Failure injection (game day)
`GET`	`/api/agent-card`	Governance: agent cards + live validation + graph
`GET`	`/api/quality`	Latest CI run, live evaluation gates, runtime self-checks
`GET`	`/health` , `/ready`	Liveness / readiness probes

Write endpoints (approve, reject, chaos, resolve) require the `X-API-Key` header when `ADMIN_API_KEY` is set. In the dashboard, enter the key under ⚙ Settings.

```
curl -X POST http://localhost:8000/api/agent/ask \
  -H "Content-Type: application/json" -H "X-Correlation-ID: demo-0001" \
  -d '{"query": "Any anomalies on my sites this month?"}'
```

🧪 Tests, evaluation & CI/CD

```
pytest --cov=app                 # 70 tests, ~95 % coverage, ~5 s, no API key needed
ruff check app tests scripts     # lint
python -m app.evaluation.run_eval # benchmark
```

Metric (evaluation suite)	Result
Routing: intent / site / time window	100 % / 100 % / 100 % (20 cases)

Metric (evaluation suite)	Result
Anomaly detection: precision / recall / F1	100 % / 100 % / 1.00 (3 seeds)
Retrieval hit@3	100 % (10 cases)
Guardrails accuracy	100 % (10 cases)
End-to-end latency (mock LLM)	p50 ≈ 5 ms

The golden sets are small and were written by the team, so these figures show that the system behaves as designed. They are not an independent benchmark. See docs/EVALUATION.md.

GitHub Actions pipeline (`.github/workflows/ci.yml`): `lint → tests + coverage gate (85 %) → evaluation → docker build → container smoke test → deploy (Hugging Face Spaces / Render) → post-deploy smoke test`

☁️ Deployment

Free, no account, no card: Cloudflare Tunnel (Windows). Double-click `start-online.bat`. It installs everything on the first run, starts the app and prints a public `https://….trycloudflare.com` link. Step by step: docs/GO_ONLINE.md.

Hugging Face Spaces. Create a Docker Space (Docker Spaces may need a paid plan), then add the `HF_TOKEN` secret and the `HF_SPACE` variable to the GitHub repo. CI deploys every green push to `main`.

Render (asks for a card to verify your identity, even on the free plan):

1. Push this repo to GitHub.
2. Render → **New → Blueprint** → select the repo (it reads `render.yaml`).
3. Add `GROQ_API_KEY` (or another provider key) in the service's **Environment** tab.
4. Open `https://<your-service>.onrender.com`.

Step by step, including the deploy hook for CI: docs/DEPLOYMENT.md

📁 Project structure

```
app/
├── agents/          # LangGraph: state, nodes (7 agents), tools, prompts, graph
├── api/             # FastAPI routes + schemas
├── core/            # observability, resilience, incidents, security, HITL actions
├── data/            # energy dataset simulation + anomaly detection
├── evaluation/      # golden sets + benchmark
├── governance/      # agent cards (JSON) + validator
├── llm/             # LLM gateway (providers, retry, circuit breaker)
├── rag/             # knowledge base (Markdown) + BM25 retriever
├── static/          # dashboard (HTML/CSS/JS, Chart.js)
└── main.py          # app entry point + correlation-ID middleware
tests/               # 70 tests
scripts/smoke_test.py
docs/                # architecture, agent card, runbook, incident report, deployment, demo
script
Dockerfile · docker-compose.yml · render.yaml · .github/workflows/ci.yml
```

📚 Documentation

Document	Content
ARCHITECTURE.md	Agents, state, data flow, design decisions
AGENT_CARD.md	Governance: system card and one card per agent
RUNBOOK.md	Incident procedures RB-001 to RB-004
INCIDENT_REPORT_INC-0001.md	Post-mortem of the LLM outage drill
GO_ONLINE.md	Put the app online from your PC with Cloudflare Tunnel
DEPLOYMENT.md	Docker, Render, CI/CD secrets
EVALUATION.md	Benchmark method and results
COURSE_MAPPING.md	Course weeks S1-S5 → implementation
DEMO_VIDEO_SCRIPT.md	5-minute demo video script
CHANGELOG.md	Versions of the system, agents and prompts

Mini-project · Energy sector · Agents in production

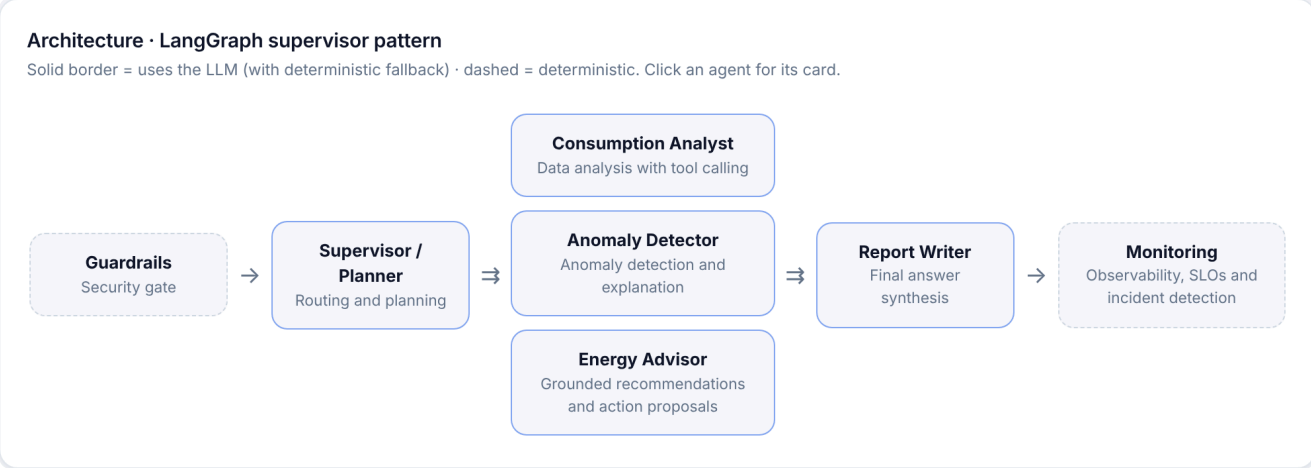
Architecture

1. Overview

EnergyWatch AI is a **multi-agent system** built with **LangGraph**, exposed through a **FastAPI** service that also serves the dashboard. One request means one *run* of the graph, and every run is identified by a **correlation ID**.

```
Browser (dashboard)  ──HTTP──▶  FastAPI  ──▶  LangGraph StateGraph  ──▶  tools / RAG / LLM gateway
        ▲                          │                  │
        └──── JSON + X-Correlation-ID ◀──── run store, metrics, incidents, audit (in memory)
                                                      └──▶ JSON logs on stdout (durable)
```

2. The graph



Architecture of the LangGraph multi-agent system (screenshot of the Governance page)

Pattern: supervisor / planner with a dispatcher. The supervisor produces a plan (an ordered list of specialists). After each node, the conditional edge `next_step()` picks the first planned agent that hasn't run yet, or moves on to `report_writer`. The graph is code-defined (`app/agents/graph.py`), and the LLM can only choose among the allowed agents, because its output is validated and sanitized.

Agents

Agent	LLM?	Responsibility	Fallback
`guardrails`	no	Injection / off-topic / length checks before any LLM call	n/a
`supervisor`	yes (structured output)	Intent, site, window, plan	Keyword planner
`consumption_analyst`	yes (tool calling)	Chooses and calls data tools, summarizes	Rule-based tool choice + template

Agent	LLM?	Responsibility	Fallback
`anomaly_detector`	yes (explanation)	Runs statistical detection, explains root causes	Root-cause table template
`energy_advisor`	yes (RAG)	Retrieves knowledge, recommends, proposes actions	Playbook + citations
`report_writer`	yes	Final Markdown answer, output guardrail	Section template
`monitor`	no	SLO check, run record, metrics, incidents	n/a

Shared state (`app/agents/state.py`)

`EnergyState` is a `TypedDict`. Most keys are overwritten by the agent that owns them. Four keys use **reducers**, so every agent can append to them:

- `completed: Annotated[list, operator.add]` lists the agents that have already run (used by the dispatcher)
- `trace: Annotated[list, operator.add]` holds one span per agent
- `errors: Annotated[list, operator.add]` collects contained failures
- `degraded: Annotated[bool, operator.or_]` becomes true if any agent fell back

3. Design decisions

Decision	Why
Numbers come from deterministic tools, never from the LLM	Avoids hallucinated kWh/MAD figures (OWASP LLM09). The LLM plans, picks tools and writes.
Every LLM agent has a deterministic fallback	The service stays useful during a provider outage (runbook RB-001).
One LLM gateway (`app/llm/gateway.py`)	Timeouts, retries, circuit breaker, chaos injection, metrics and run accounting live in one place.
Mock provider	Tests and CI are free, fast and deterministic, and the demo works offline.
BM25 retriever in pure Python	No vector DB to operate; 29 chunks only. The `search()` interface can be swapped for a vector store.
Statistical anomaly detection (robust z-score)	Explainable, and fast (≈20 ms for 16 meters × 720 h). It is evaluated against ground truth.
Human approval for medium/high-risk actions	Excessive agency (OWASP LLM06); clinical and data-center operations are safety-critical.
In-memory telemetry + JSON logs	Simple for a single instance; logs are the durable source of truth on Render. To scale out, export to Prometheus / OpenTelemetry.

4. Observability: how the correlation ID flows

1. `correlation_middleware` (`app/main.py`) reads `X-Correlation-ID`, or creates `ewa-<20 hex>`. Unsafe values are replaced.
2. The ID is stored in a `ContextVar` and returned in the response header.
3. `run_agent()` puts it into the graph state and the run store.
4. `traced()` re-binds it inside every node and records a `Span` (agent, duration, status, tool calls, error).
5. The `JsonFormatter` adds it to **every log line**. Incidents, audit entries and proposed actions store it too.

So `docker logs energywatch | grep <correlation-id>` returns the whole story of one request.

5. Reliability

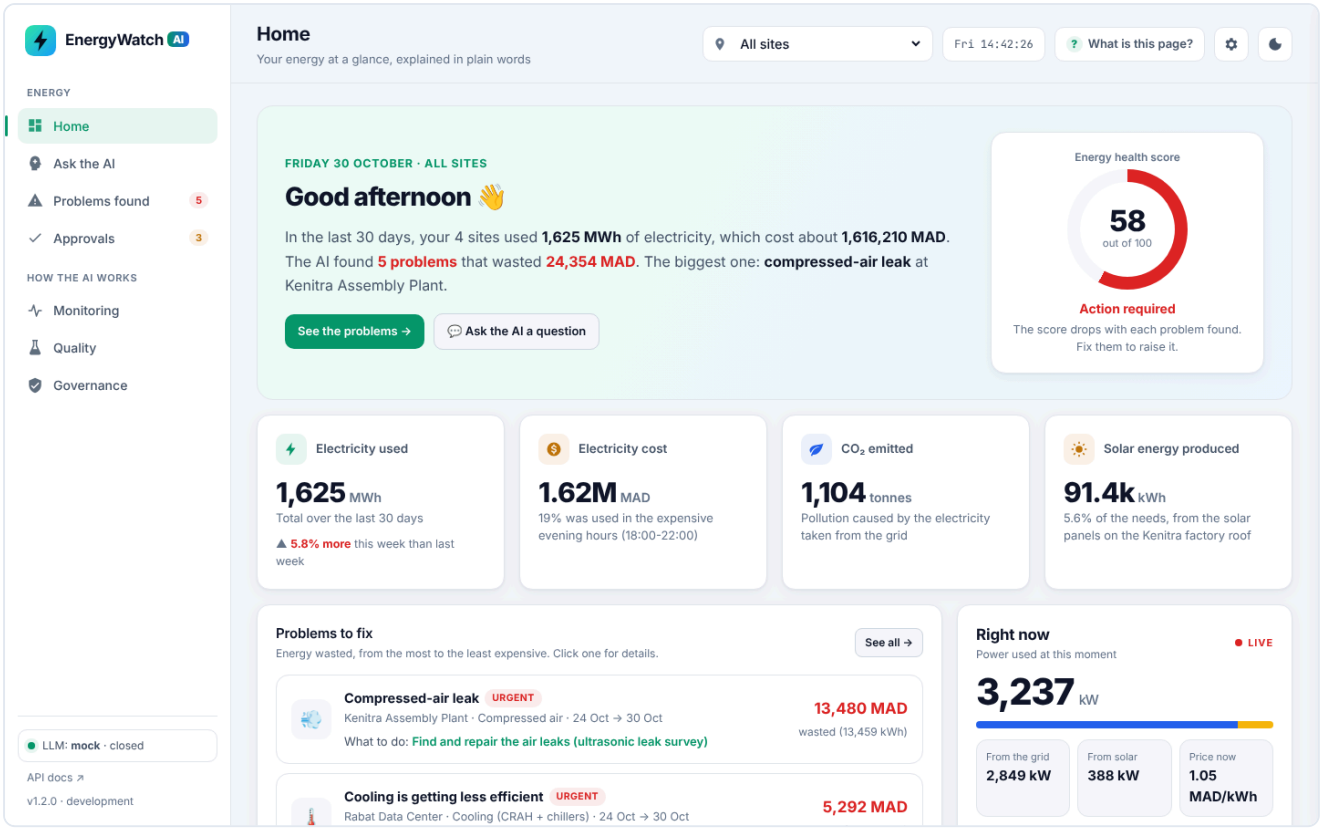
```
agent ──► gateway._call()
            ├─ breaker open?  ──► LLMUnavailable (short-circuit, no waiting)
            ├─ chaos hooks (outage / latency)
            ├─ retry_call(attempts = LLM_MAX_RETRIES + 1, exponential backoff)
            ├─ failure ──► breaker.record_failure() ──(threshold)──► OPEN ──► incident RB-001
            └─ success ──► breaker.record_success() ──(was half-open)──► CLOSED ──► RB-001
resolved
agent catches LLMUnavailable ──► deterministic fallback, span status = degraded
```

6. Data

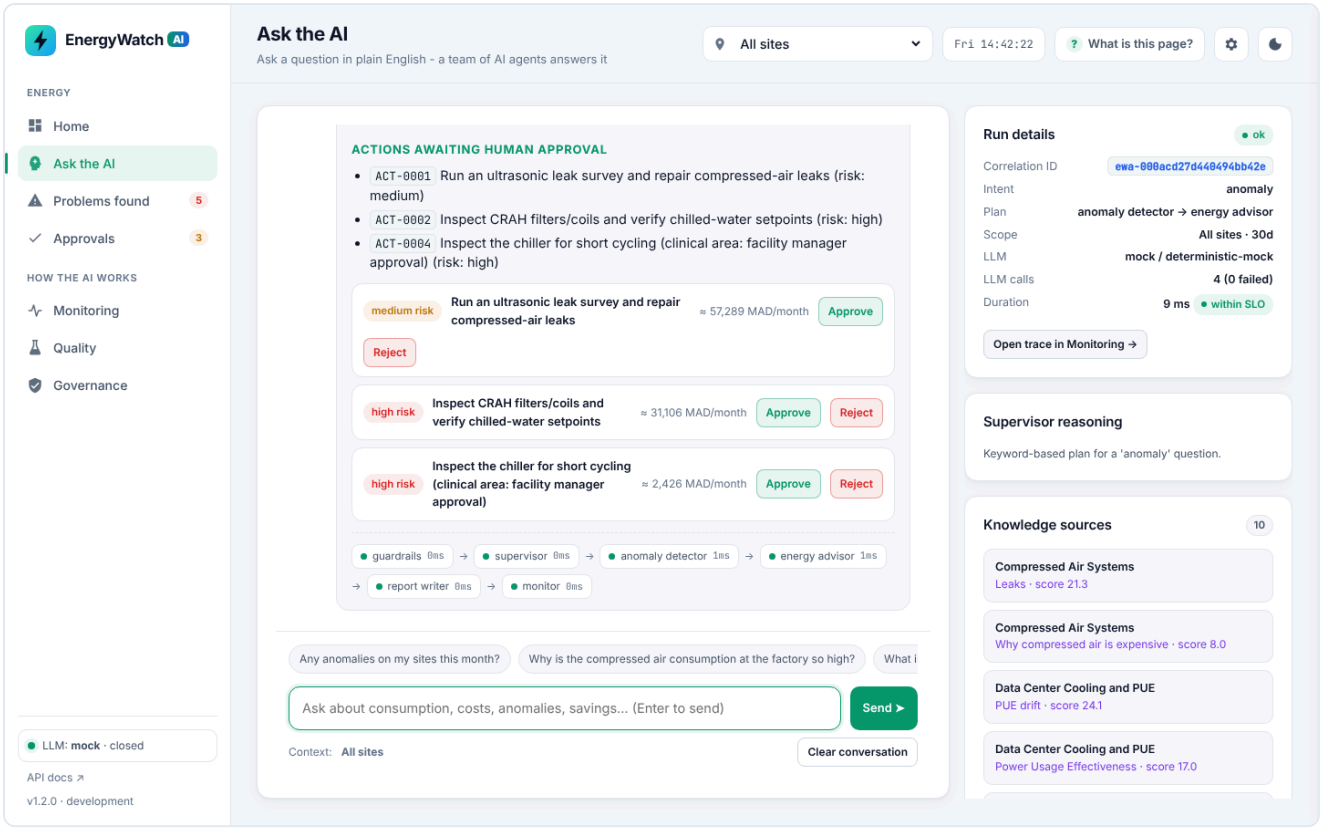
`app/data/energy_data.py` simulates 4 sites, 16 meters and 30 days of hourly data, with realistic profiles (occupancy, shifts, outdoor temperature, solar curve) and 5 injected anomalies. In production, replace `EnergyDataset` with a connector to the metering platform; the tools and agents won't change.

Anomaly detection, per meter: 1. Baseline = median for each `(working day?, hour)` bucket over the first 14 days. 2. Robust scale = MAD of the training residuals (with a floor). 3. Hours with z > 3.5 and a +12 % deviation are flagged, then grouped (gaps ≤ 16 h bridged). 4. Noise filter: ≥ 3 hours or z ≥ 12, and cost ≥ 100 MAD. 5. Pattern classification: weekend, drift / sustained, night, spike. 6. Each event gets excess kWh, cost at the time-of-use tariff, and a severity.

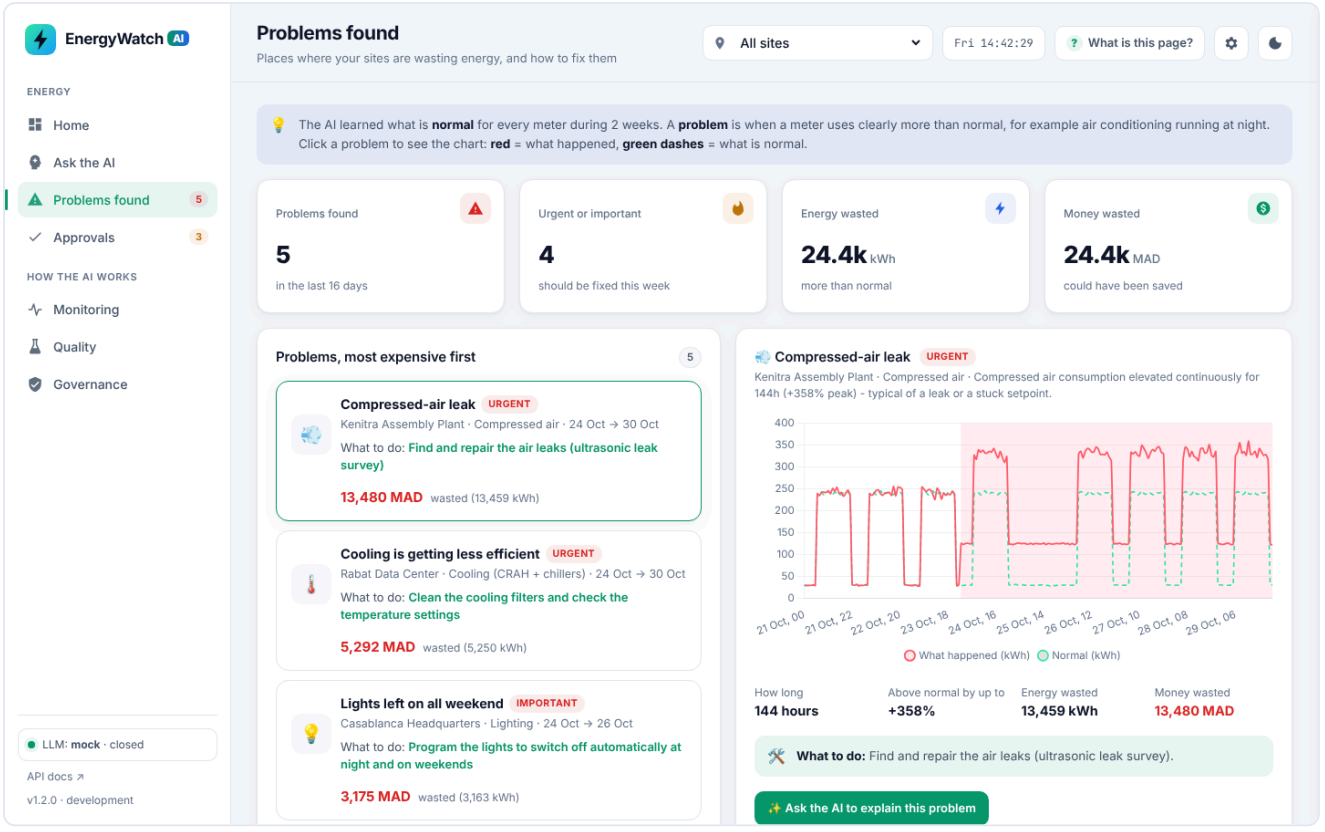
Dashboard screenshots



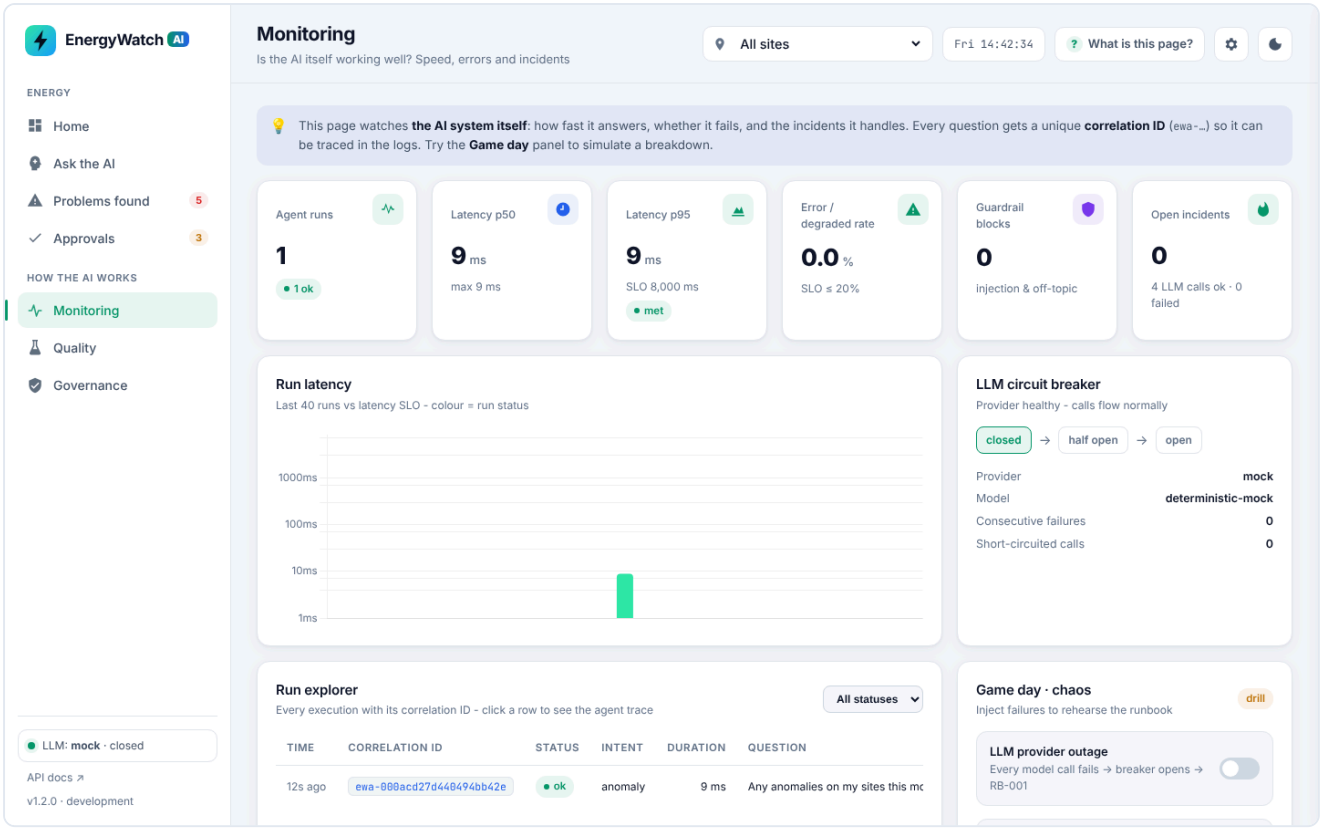
Home: one-sentence summary, energy health score, problems to fix



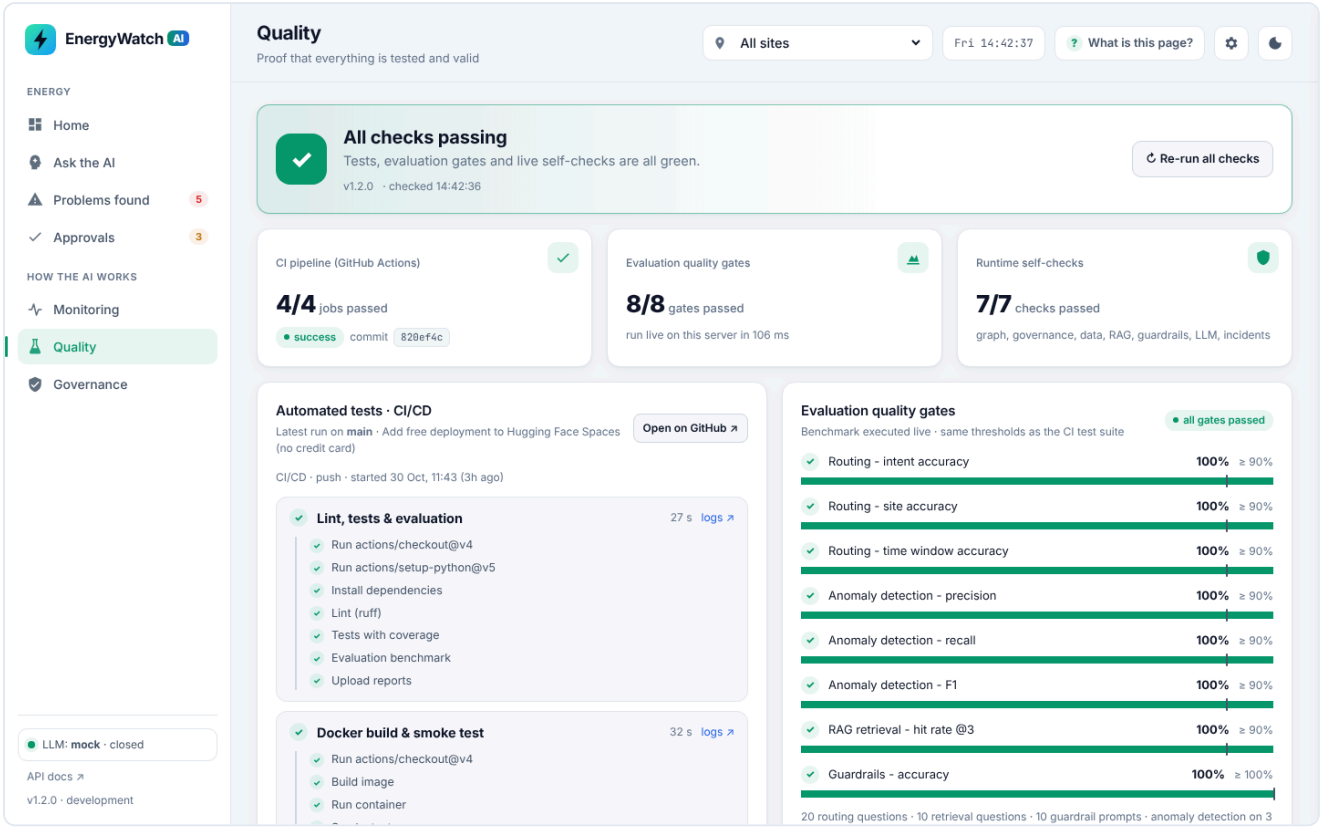
Ask the AI: multi-agent answer with plan, sources, approvals and trace



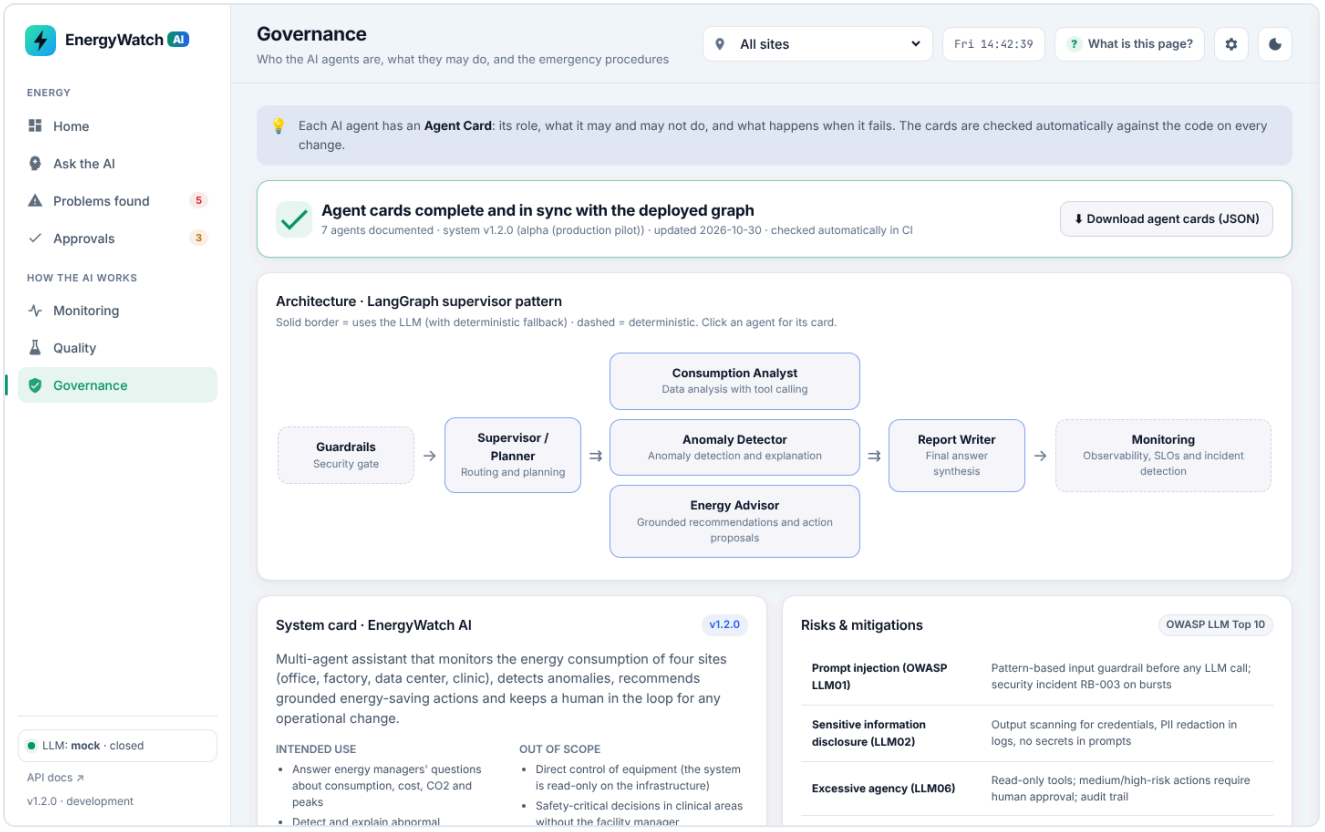
Problems found: actual consumption vs. normal baseline



Monitoring: runs, latency SLO, circuit breaker, incidents



Quality: CI pipeline, evaluation gates and live self-checks



Governance: agent cards, architecture and runbooks

Evaluation & Benchmarking

Run it with: `python -m app.evaluation.run_eval --json reports/eval.json`. It runs in CI on every push, and the report is uploaded as an artifact. Quality gates are enforced by `tests/test_governance_eval.py::test_evaluation_quality_gates`.

Results (v1.0.0, deterministic mock LLM)

Component	Metric	Result	Gate
Supervisor routing	Intent accuracy (20 questions)	**100 %**	≥ 90 %
	Site accuracy	**100 %**	≥ 90 %
	Time-window accuracy	**100 %**	-
Anomaly detector	Precision / recall / F1 (3 seeds × 5 injected anomalies)	**1.00 / 1.00 / 1.00**	F1 ≥ 0.9
	Robustness: 30 seeds × 7 calendar alignments	**0 mismatches / 210**	tested
RAG retriever	Hit rate @3 (10 questions)	**100 %**	≥ 90 %
Guardrails	Accuracy (7 attacks / off-topic + 3 legitimate)	**100 %**	= 100 %
End-to-end	Latency p50 / p95 (mock LLM, 10 runs)	≈ 5 ms / ≈ 50 ms	SLO 8 s

Method

- **Routing:** `app/evaluation/golden_set.json → routing`. Each question has an expected intent, and optionally a site and window. It is evaluated on the deterministic planner that also serves as the fallback when the LLM is down.
- **Anomaly detection:** the simulation injects 5 anomalies with a known meter and pattern. A detection counts as a true positive only if both the meter **and** the pattern classification match.
- **Retrieval:** a hit means the expected document appears in the top 3 BM25 results.
- **Guardrails:** injection and off-topic prompts must be blocked; legitimate energy questions must pass.

Limitations (honest reading of the numbers)

- The golden sets are **small and were written by the team** while building the rules, so 100 % shows the system behaves as designed. It is not a measure of generalization. More varied phrasings would lower the routing accuracy of the keyword planner. That is exactly why a real LLM planner is used when a key is configured, with the rules only as a fallback.
- The anomaly data is **simulated**. Real meters have more noise, missing data and legitimate schedule changes.
- With a real LLM, answers are non-deterministic. We evaluate the **structure** (plan, tools called, citations present, numbers taken from the tools) rather than the exact wording.

Ideas for v1.1

- Add 100+ paraphrased questions, including French and Darija, to the routing set.
- LLM-as-judge for answer faithfulness: every number in the answer must appear in the tool results.
- Evaluate on a public metering dataset with labelled anomalies.

Course mapping: where each week's concepts live in the code

Week	Topic	Where in EnergyWatch AI
S1	Agentic loop & model taxonomy	`app/agents/graph.py` : plan → act (tools) → observe → answer; multi-provider gateway `app/llm/gateway.py`
	Tool calling: how the agent chooses a tool	`consumption_analyst_node` uses `bind_tools(ANALYST_TOOLS)` and the LLM picks the tools; `app/agents/tools.py` validates the arguments
	Stable prompts, structured outputs, routing	Versioned prompts `app/agents/prompts.py` ; `RoutingDecision` Pydantic schema + `with_structured_output` ; `_sanitize_decision`
	Memory & state	`EnergyState` with reducers (`trace` , `completed` , `errors` , `degraded`); run store; chat history in the dashboard
S2	RAG fundamentals	`app/rag/` : 7 Markdown guides, used by `energy_advisor_node` with citations
	Advanced RAG: chunking & search	Heading-aware chunking with overlapping windows, BM25 scoring, domain query expansion (`SYNONYMS`), relevance threshold
	Structured extraction	Raw hourly meter data turned into structured `AnomalyEvent` records (pattern, cost, severity)
	LLM agent security (OWASP)	`app/core/security.py` : LLM01 injection, LLM02 leakage (output + log redaction), LLM06 least privilege + HITL, LLM10 rate limit and length
S3	Agentic architectures: planning	Supervisor / planner pattern + dispatcher `next_step()`
	Reliability engineering	Timeouts, retries with backoff, deterministic fallbacks, degraded mode
	Resilience in the architecture	`app/core/resilience.py` : circuit breaker (closed / open / half-open), chaos controller
	Evaluation & benchmarking	`app/evaluation/` : golden sets, precision/recall, hit@3, latency, CI quality gates
S4	Multi-agent architectures & patterns	7 agents in one LangGraph `StateGraph` (supervisor pattern)
	Multi-agent framework	LangGraph (conditional edges, reducers, compiled graph, Mermaid export)
	Business agents, human-in-the-loop & audit	`app/core/actions.py` : risk-based approval policy, Approve/Reject in the UI, append-only audit trail
	Multimodal agents	Not in scope for v1.0; see the roadmap in CHANGELOG (meter photo / nameplate reading)

Week	Topic	Where in EnergyWatch AI
S5	Agent observability	Correlation IDs, JSON logs, spans, Prometheus `/metrics`, Monitoring page
	Automated tests & agentic CI/CD	70 pytest tests, `.github/workflows/ci.yml`, smoke tests before and after deploy
	Governance, agent cards & versioning	`app/governance/agent_cards.json` validated in CI, `docs/AGENT_CARD.md`, `CHANGELOG.md`
	Deployment: containerization & cloud	`Dockerfile`, `docker-compose.yml`, `render.yaml`, `docs/DEPLOYMENT.md`

Deployment Guide

1. Run locally (no Docker)

```
python -m venv .venv
.venv\Scripts\activate            # Windows   (macOS/Linux: source .venv/bin/activate)
pip install -r requirements-dev.txt
copy .env.example .env            # Windows   (macOS/Linux: cp .env.example .env), then add a
key (optional)
uvicorn app.main:app --reload
```

Open http://localhost:8000

2. Docker

```
docker build -t energywatch-ai .
docker run -p 8000:8000 --env-file .env energywatch-ai
# or
docker compose up --build
```

About the image: - **Multi-stage build:** dependencies are installed in a builder stage, and the runtime image only contains the virtualenv and `app/`. - **Non-root user** (`energywatch`, uid 10001). - `HEALTHCHECK` on `/health`. - **Honours** `$PORT` (needed by Render); defaults to 8000. - **Single worker** on purpose: the telemetry stores are in memory.

Check it: `python scripts/smoke_test.py http://localhost:8000` (9 checks).

3. Render (cloud)

Option A: Blueprint (recommended)

1. Push the repository to GitHub.
2. Sign in at https://render.com with GitHub.
3. **New + → Blueprint** → choose the repository. Render reads `render.yaml` and creates the `energywatch-ai` web service (Docker runtime, free plan, health check `/health`).
4. In the service's **Environment** tab, set `GROQ_API_KEY` (free at https://console.groq.com) or `GOOGLE_API_KEY` (free at https://aistudio.google.com). If you set neither, the app runs in mock mode.
5. `ADMIN_API_KEY` is generated by Render. Copy it from the Environment tab and paste it in the dashboard under ⚙ **Settings** to approve actions and run drills.
6. Wait for the deploy to finish, then open `https://energywatch-ai-xxxx.onrender.com`.

Option B: Manual web service

New + → **Web Service** → your repo → Runtime **Docker** → Health check path `/health` → add the environment variables above → Create.

> **Free plan note:** the service sleeps after 15 minutes of inactivity, and the first request after that takes about 30-60 s. Open the URL a minute before your demo.

3 bis. Cloudflare Tunnel from your PC (free, no account, no card)

See GO_ONLINE.md: double-click `start-online.bat` to get a public `https://….trycloudflare.com` link to the app running on your PC.

3 ter. Hugging Face Spaces

Docker Spaces may require a paid Hugging Face plan. If your account allows them:

One-time setup (≈ 5 minutes): 1. Create a free account at https://huggingface.co/join. 2. Create a Space at https://huggingface.co/new-space: - Space name: `energywatch-ai` - SDK: **Docker** → *Blank* template - Hardware: **CPU basic · Free** - Visibility: Public 3. Create a token at https://huggingface.co/settings/tokens → *Create new token* → type **Write**. Copy it. 4. In the Space → **Settings → Variables and secrets**, add the secrets `GROQ_API_KEY` (optional) and `ADMIN_API_KEY` (any password you choose). 5. In the GitHub repo → **Settings → Secrets and variables → Actions**: - **Secrets** tab → *New repository secret*: `HF_TOKEN` = the token from step 3 - **Variables** tab → *New repository variable*: `HF_SPACE` = `<your-hf-username>/energywatch-ai` 6. Re-run the latest workflow (Actions → CI/CD → *Re-run all jobs*), or push any commit.

From then on, every push to `main` that passes the tests is deployed automatically by the `deploy-huggingface` job. The app is available at `https://<your-hf-username>-energywatch-ai.hf.space`.

> Free Spaces sleep after 48 h without visits; the first visit wakes them up (≈ 1 min).

4. CI/CD with GitHub Actions

`.github/workflows/ci.yml` runs on every push and pull request:

Job	Steps
`test`	ruff lint → pytest with coverage (fails under 85 %) → evaluation benchmark → upload reports
`docker`	build the image → run the container → `scripts/smoke_test.py` → print logs
`deploy` (push to `main` only)	trigger the Render deploy hook → post-deploy smoke test against the live URL
`deploy-huggingface` (push to `main` only)	push the app to the Hugging Face Space → post-deploy smoke test

Optional configuration (repo → Settings → Secrets and variables → Actions):

Name	Type	Value
`RENDER_DEPLOY_HOOK_URL`	Secret	Render → service → Settings → *Deploy Hook* URL

Name	Type	Value
`RENDER_SERVICE_URL`	Variable	`https://energywatch-ai-xxxx.onrender.com`
`HF_TOKEN`	Secret	Hugging Face token with **Write** access
`HF_SPACE`	Variable	`<hf-username>/energywatch-ai`

Without them, the deploy jobs are skipped with a notice, and Render's own *auto-deploy on push* still works. If you use the deploy hook, turn auto-deploy off in Render so each commit deploys only once, after CI passes.

5. Configuration reference

Variable	Default	Description
`LLM_PROVIDER`	`auto`	`auto`, `groq`, `gemini`, `openai`, `anthropic` or `mock`
`LLM_MODEL`	provider default	Override the model name
`GROQ_API_KEY` / `GOOGLE_API_KEY` / `OPENAI_API_KEY` / `ANTHROPIC_API_KEY`	-	Provider keys
`LLM_TIMEOUT_S`	`20`	Per-call timeout
`LLM_MAX_RETRIES`	`2`	Retries per call (exponential backoff)
`CIRCUIT_BREAKER_THRESHOLD`	`3`	Consecutive failures before the breaker opens
`CIRCUIT_BREAKER_RESET_S`	`30`	Cool-down before the half-open probe
`SLO_LATENCY_MS`	`8000`	Latency SLO used by the monitoring agent
`ADMIN_API_KEY`	empty	Protects approve/reject/chaos/resolve when set
`RATE_LIMIT_PER_MINUTE`	`30`	Per-client limit on `/api/agent/ask`
`LOG_LEVEL`	`INFO`	JSON logs on stdout
`ENVIRONMENT`	`development`	Shown in the dashboard and `/ready`

Put EnergyWatch AI online with Cloudflare Tunnel (free, no card)

Cloudflare Tunnel publishes the app running on your PC at a public `https://….trycloudflare.com` address. You don't need a Cloudflare account, a credit card or Docker.

> Cloudflare Pages and Workers can't host this app: it is a Python server with AI agents. The tunnel runs it on your PC and Cloudflare carries the traffic.

One-time preparation

1. Install **Python 3.12** from https://www.python.org/downloads/ and tick **"Add python.exe to PATH"**.
2. In the project folder, copy `.env.example` to `.env`, open it with Notepad and set:
   `GROQ_API_KEY=your-groq-key (free at https://console.groq.com, optional)`
   `ADMIN_API_KEY=choose-a-password (important when the link is public)`

Every time you want to go online

1. Double-click **`start-online.bat`** in the project folder.
2. Wait. The first run takes a few minutes, because it installs the libraries and downloads Cloudflare Tunnel. Two windows open: - **"server"**: the app itself. Keep it open. - **"online with Cloudflare Tunnel"**: after a few seconds it prints a box containing your link: `Your quick Tunnel has been created! Visit it at: https://some-random-words.trycloudflare.com`
3. Open that link on any device, or send it to your teacher.
4. In the dashboard, click ⚙ **Settings** and enter your `ADMIN_API_KEY`, so you can approve actions and run the incident drill.

To go offline, close both windows, or press **Ctrl+C** in each.

Good to know

- The link only works **while your PC is on and both windows are open**.
- The link **changes each time** you start the tunnel, so send the new one.
- If Windows SmartScreen warns about the `.bat` file, click **More info → Run anyway**.
- To use the app only on your PC (no public link), double-click **`start-local.bat`**.
- Check the live link from another PC: `python scripts/smoke_test.py https://your-link.trycloudflare.com`

Troubleshooting

Problem	Fix
"Python 3.12 was not found"	Install Python 3.12 (see above), then run the file again
The server window shows a red error	Send a screenshot of it; often a typo in `.env`

Problem	Fix
No link appears	Check your internet connection; close and run `start-online.bat` again
"Missing or invalid X-API-Key" when approving	Enter your `ADMIN_API_KEY` in ⚙ Settings